

PROYECTO: ARQUITECTURA DE SOFTWARE

DIAGRAMAS DE CLASES UML

La Arquitectura del Software

Una guía visual para modelar la estructura
estática de sistemas complejos.



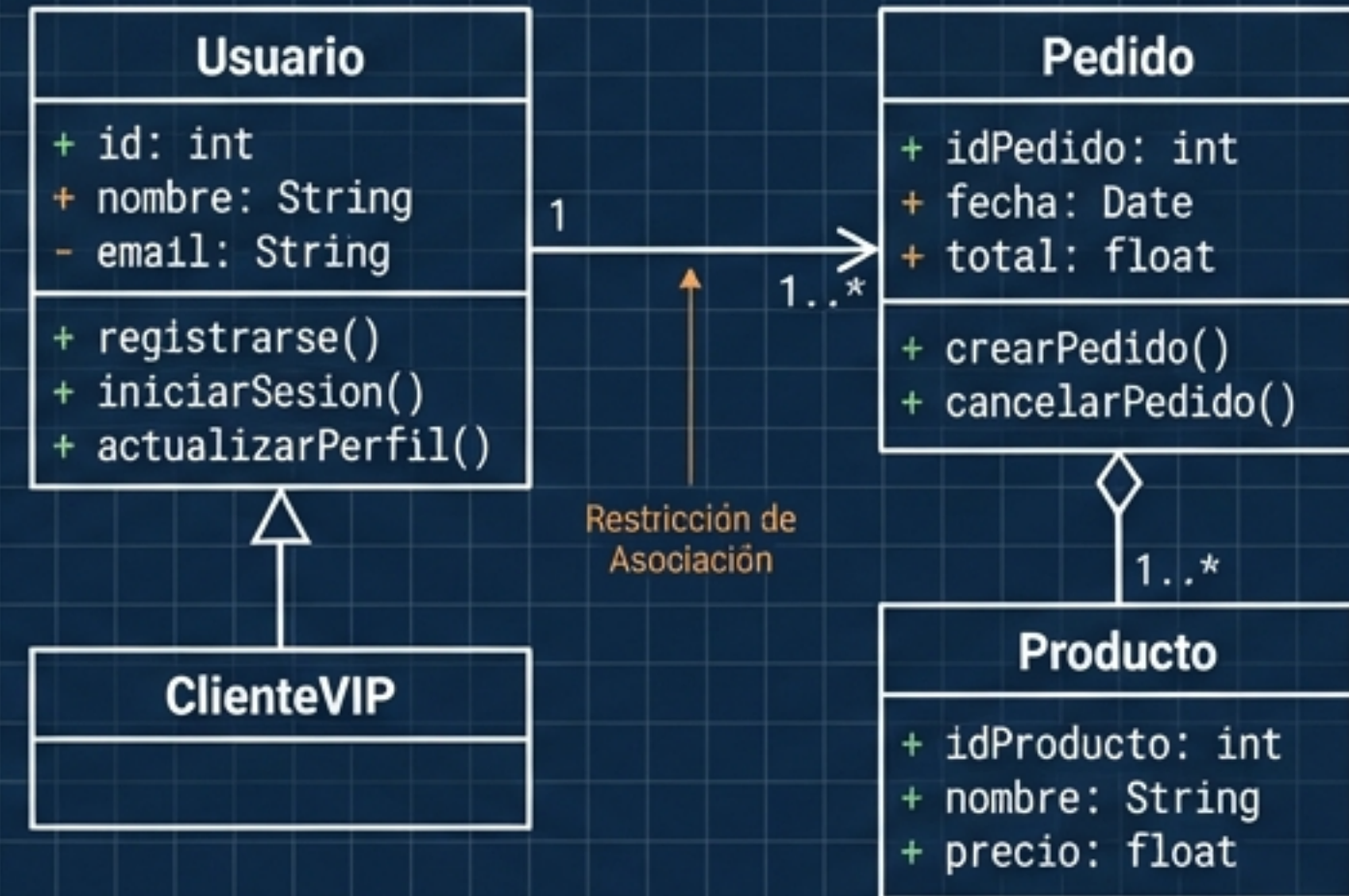
Basado en el estándar UML (Lenguaje Unificado de Modelado)

El Plano Estático del Sistema

DINÁMICO (Comportamiento)



ESTÁTICO (Estructura)



El **Diagrama de Clases** es el pilar fundamental del modelado orientado a objetos. A diferencia de los diagramas de comportamiento (que muestran qué sucede), este diagrama estructural muestra **qué existe**.

Es el **plano** (blueprint) del sistema. Modela los objetos, sus relaciones y los **servicios** que proveen.

Propósito y Aplicación en el Ciclo de Vida



1. Análisis y Diseño

Comprender los requisitos del dominio e identificar sus componentes antes de escribir código.



2. Comunicación

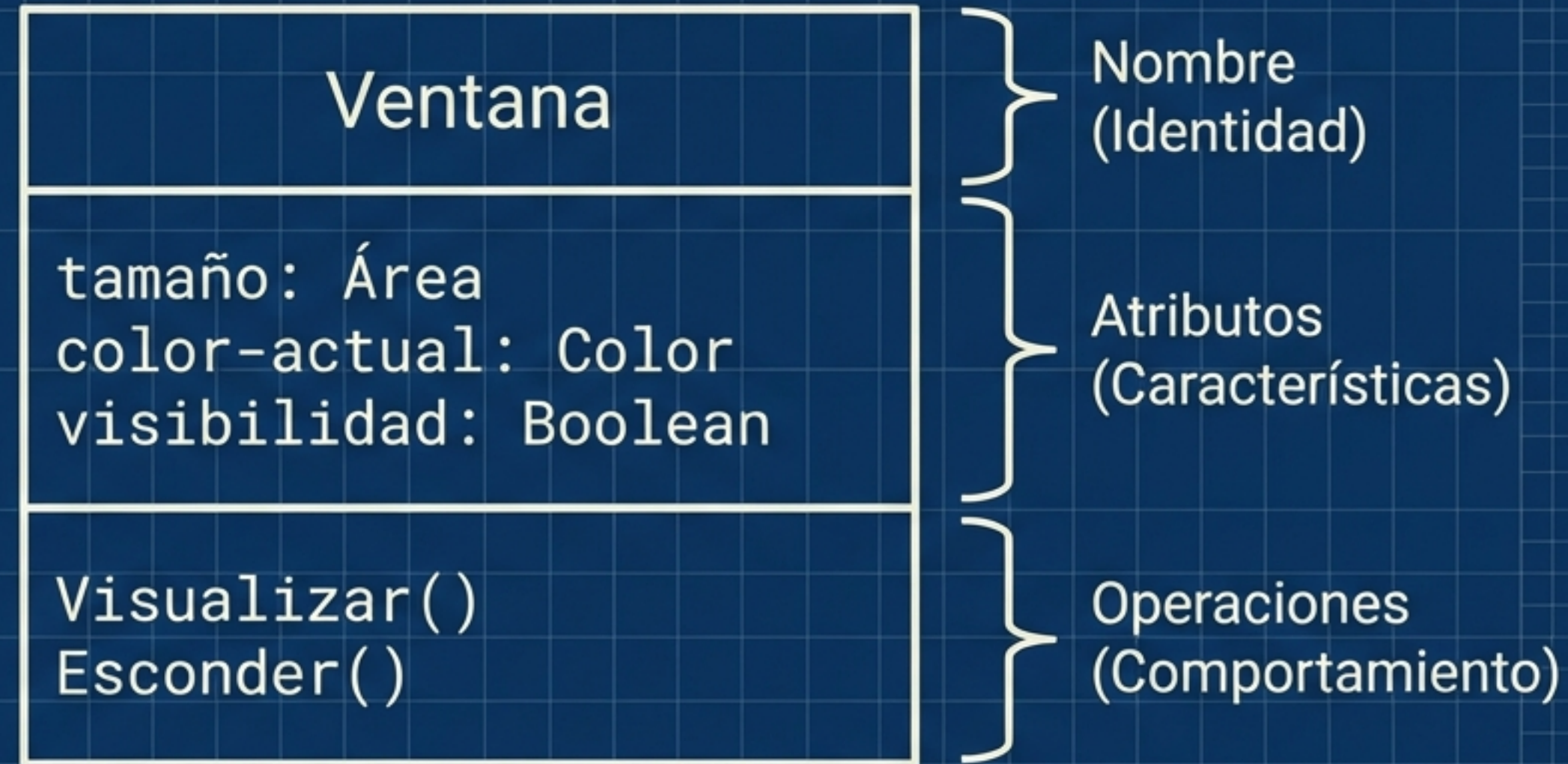
Un lenguaje común para discutir jerarquías y colaboraciones con stakeholders y el equipo.



3. Implementación

Guía directa para la generación de código (Java, C++) convirtiendo modelos en software real.

La Unidad Fundamental: Anatomía de una Clase



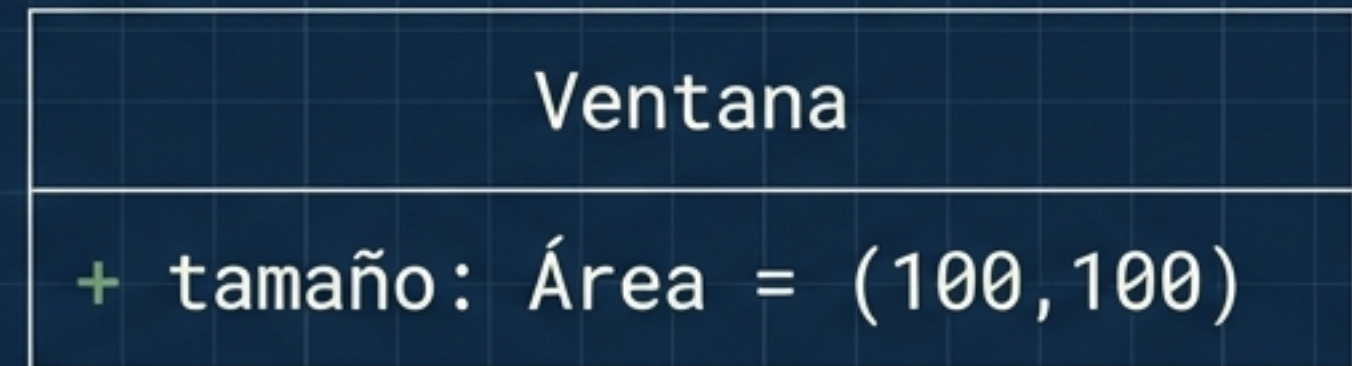
Una clase describe un conjunto de objetos con características y comportamiento idénticos.

Definiendo los Datos: Atributos y Visibilidad

`[visibilidad] nombre [multiplicidad] : [tipo] = [valor-inicial]`

Leyenda de Visibilidad

- +** Pública (Visible para todos)
- Privada (Visible solo para la clase)
- #** Protegida (Visible para subclases)
- ~** Paquete (Visible en el paquete)

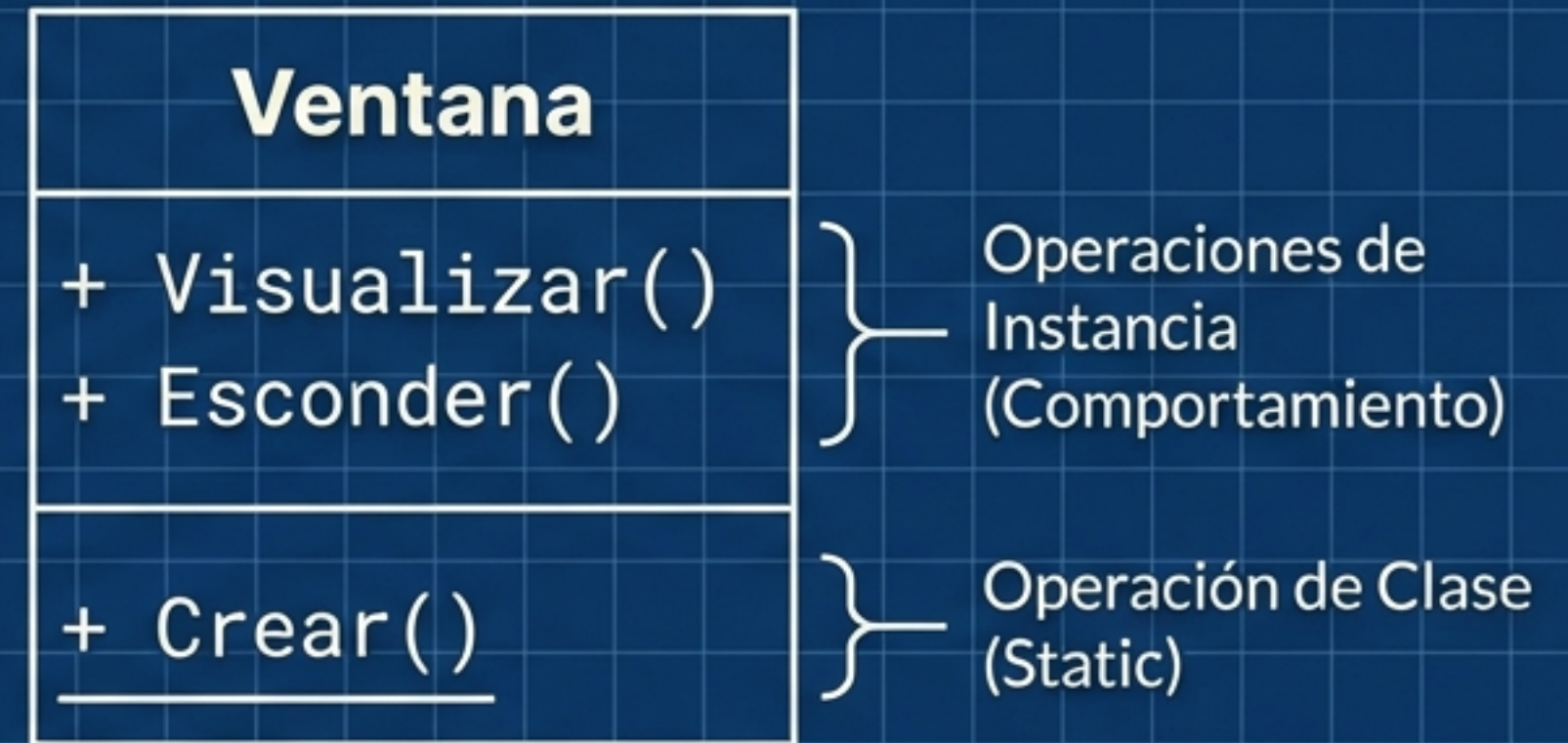


Modelando el Comportamiento: Operaciones

[visibilidad] nombre(parámetros) : [tipo-devuelto]

Ámbito:

1. **Instancia:** Operaciones normales que requieren un objeto.
2. **Ámbito de Clase (Static):** Se representan subrayadas. No requieren una instancia.

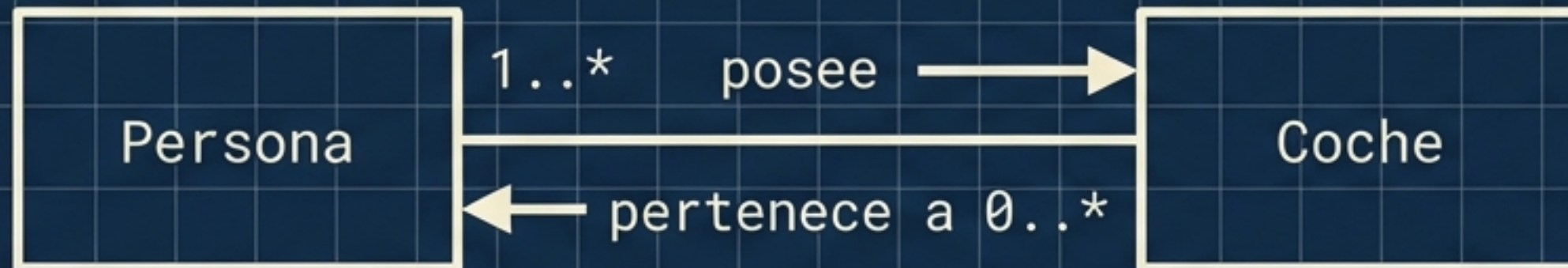


El Mapa de Relaciones

Tipo	Línea / Símbolo	Significado
Asociación		Conexión estructural
Agregación		Todo-Parte (Débil)
Composición		Todo-Parte (Fuerte)
Generalización		Herencia (Es un...)
Dependencia		Uso temporal (Usa un...)

“Una relación es una conexión semántica entre elementos del modelo.”

Asociación: Conexiones Estructurales



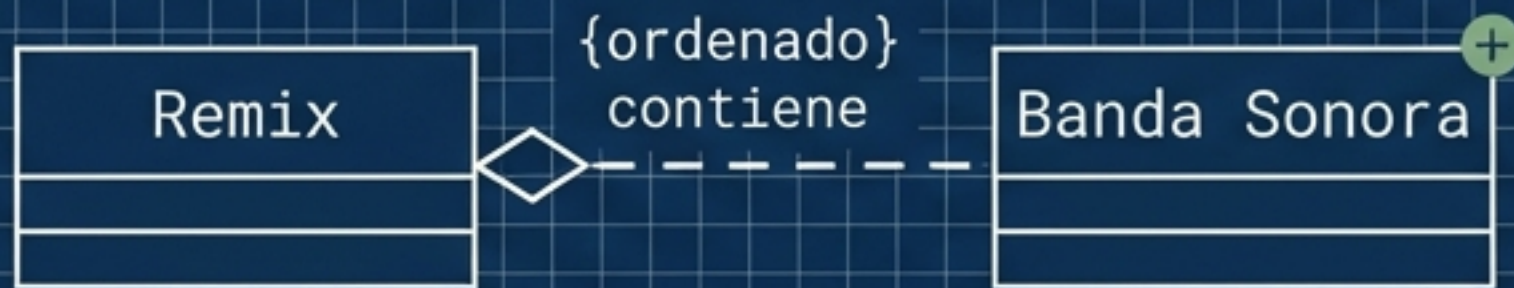
Multiplicidad

1 : Exactamente uno
0..1 : Cero o uno
0..* : Cero a muchos
1..* : Uno o más

- **Navigabilidad:** La flecha indica la dirección del conocimiento entre clases.

La Jerarquía Todo-Parte: Agregación vs. Composición

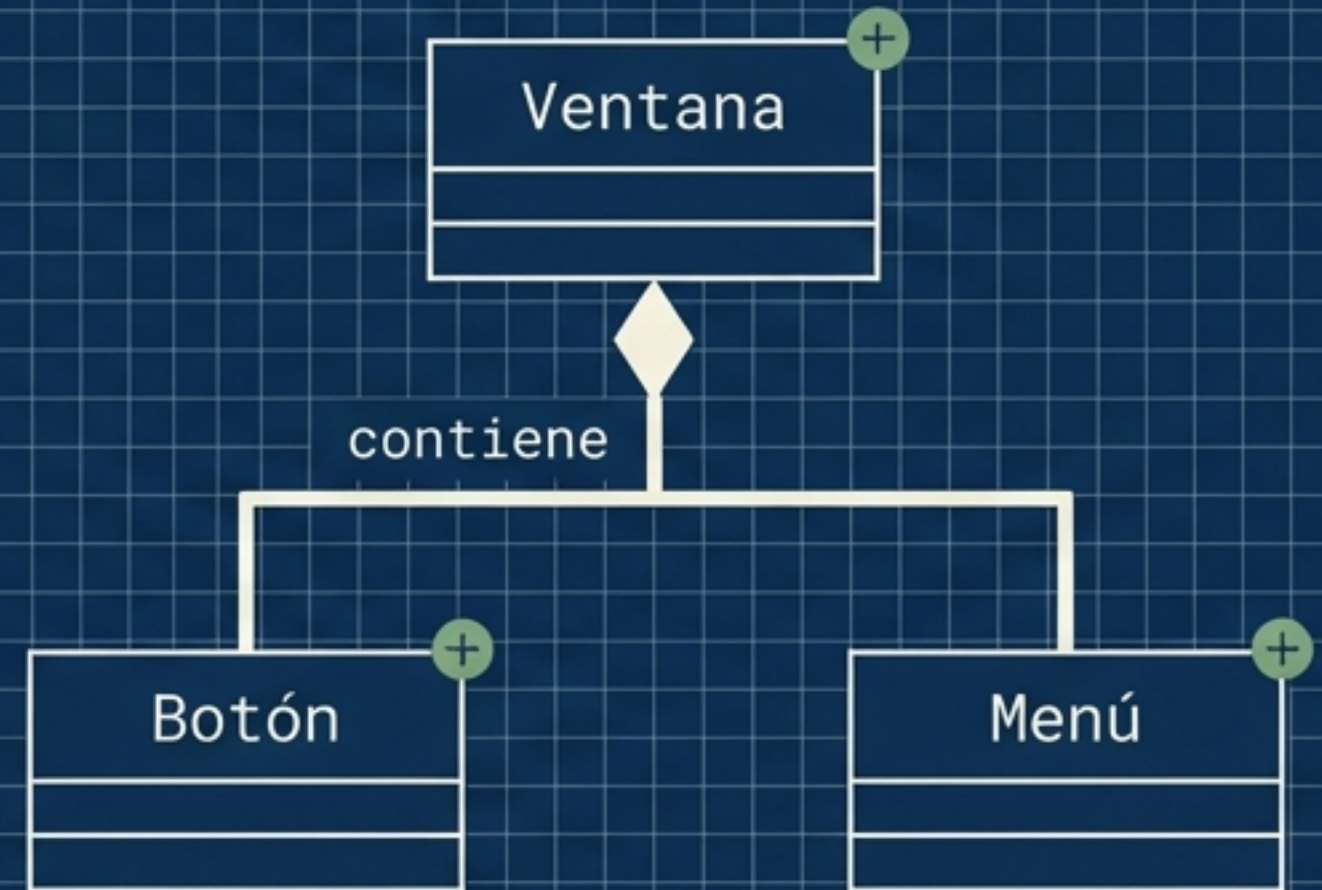
Agregación (Débil)



Las partes existen sin el todo.

Vínculo débil.

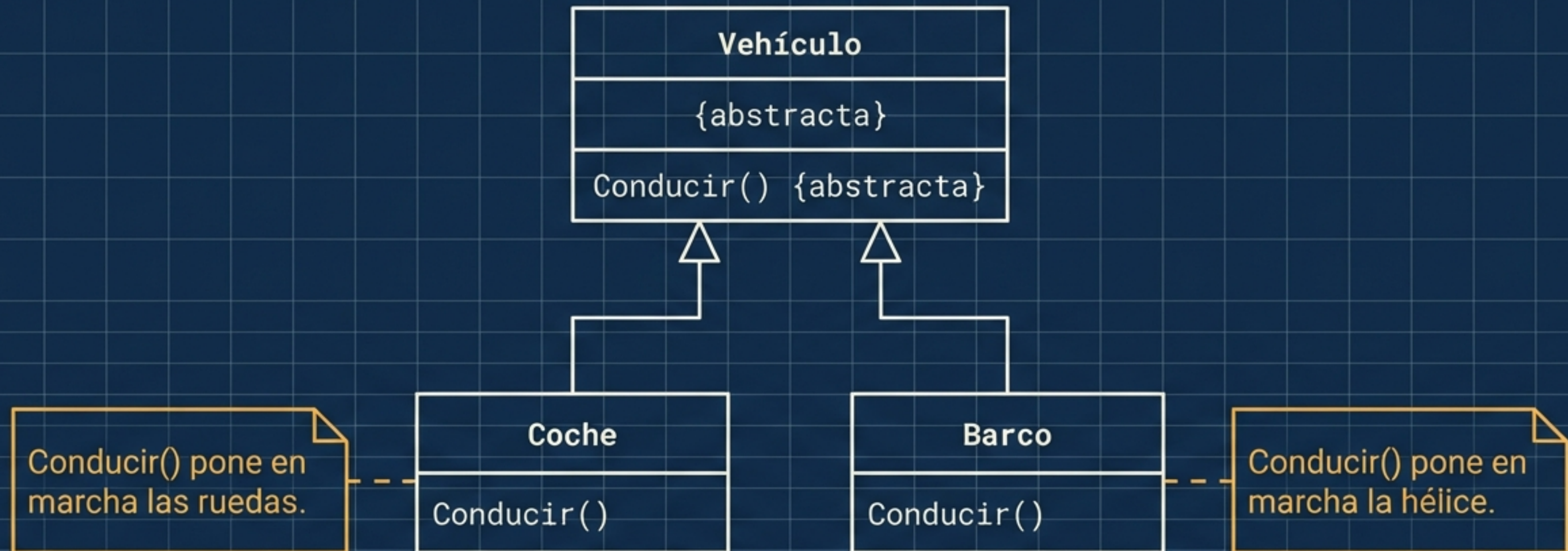
Composición (Fuerte)



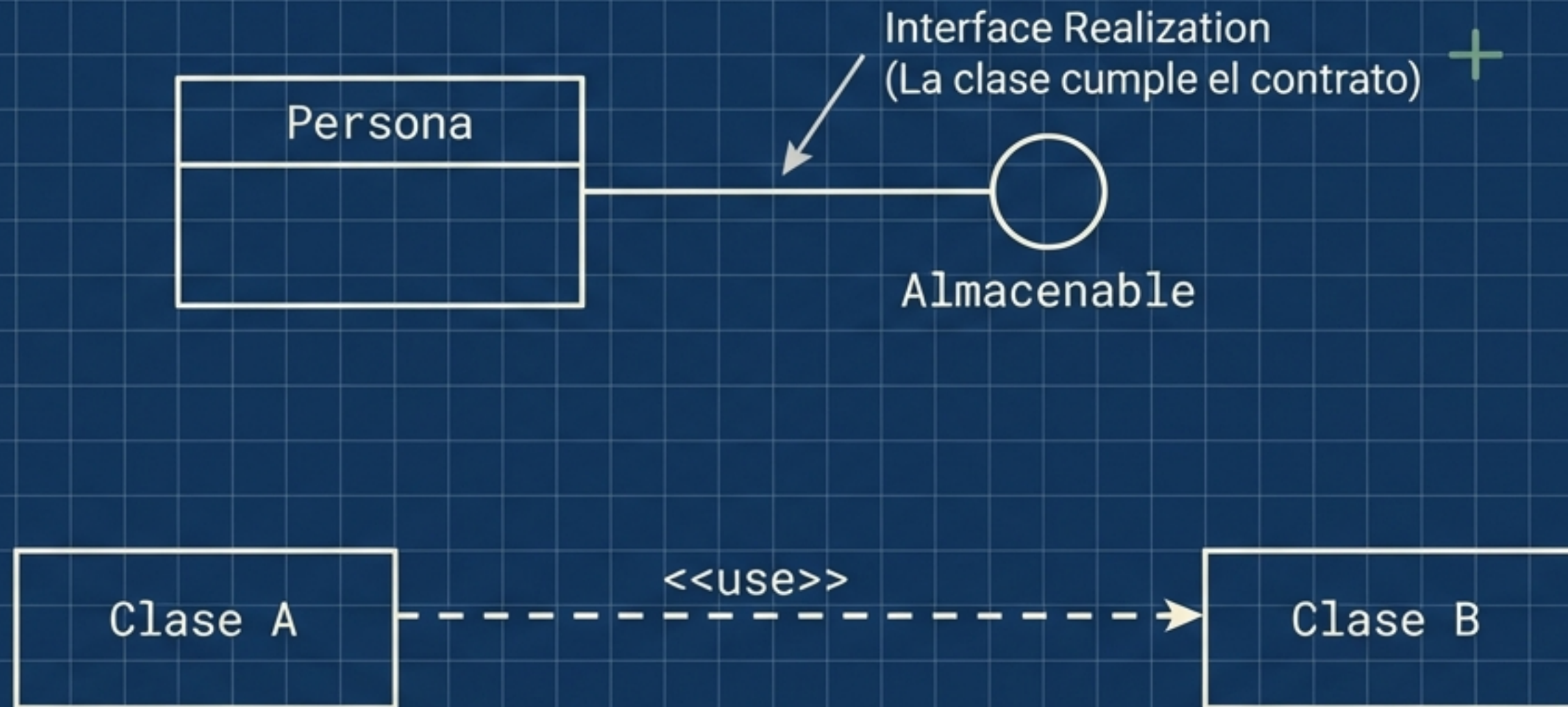
Dependencia Existencial: Si muere el todo, mueren las partes.

Vínculo fuerte.

Generalización: La Relación “Es un...”

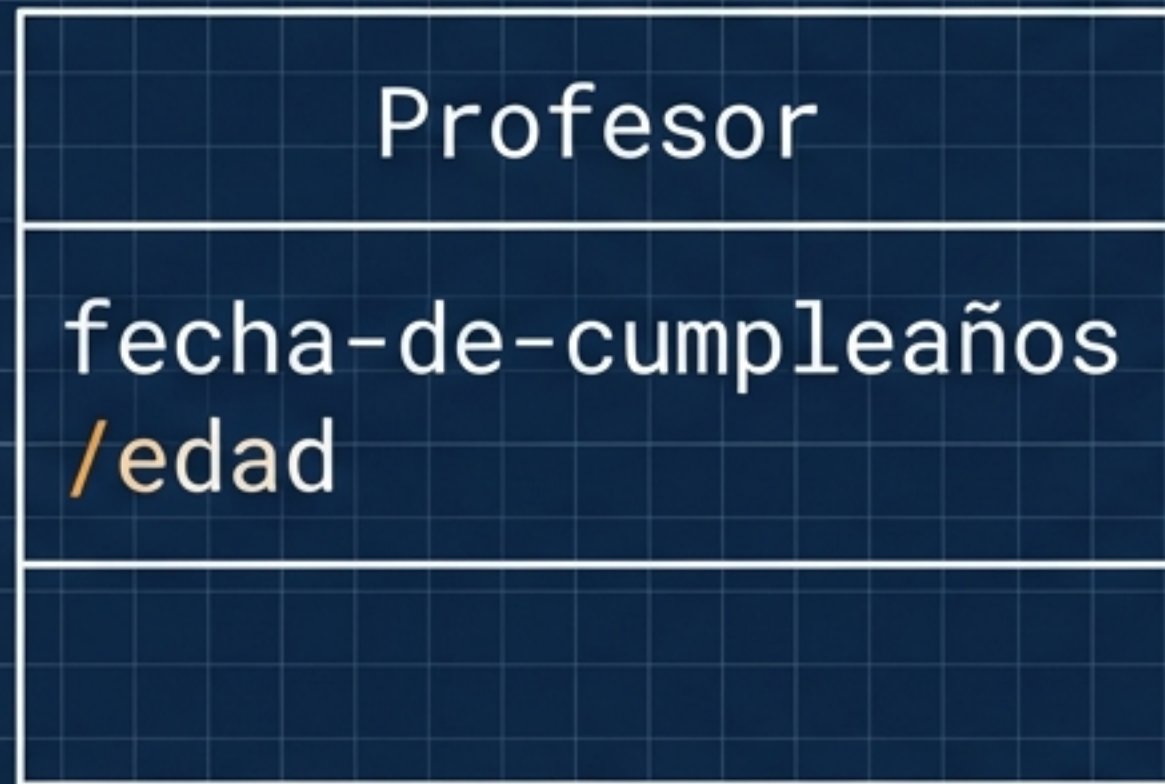


Dependencia y Contratos (Interfaces)



- **Dependencia (Línea discontinua):** Relación “usa un”. El cambio en el independiente afecta al dependiente.
- **Interface (Círculo):** Un contrato de comportamiento sin implementación.

Refinando el Modelo: Elementos Avanzados

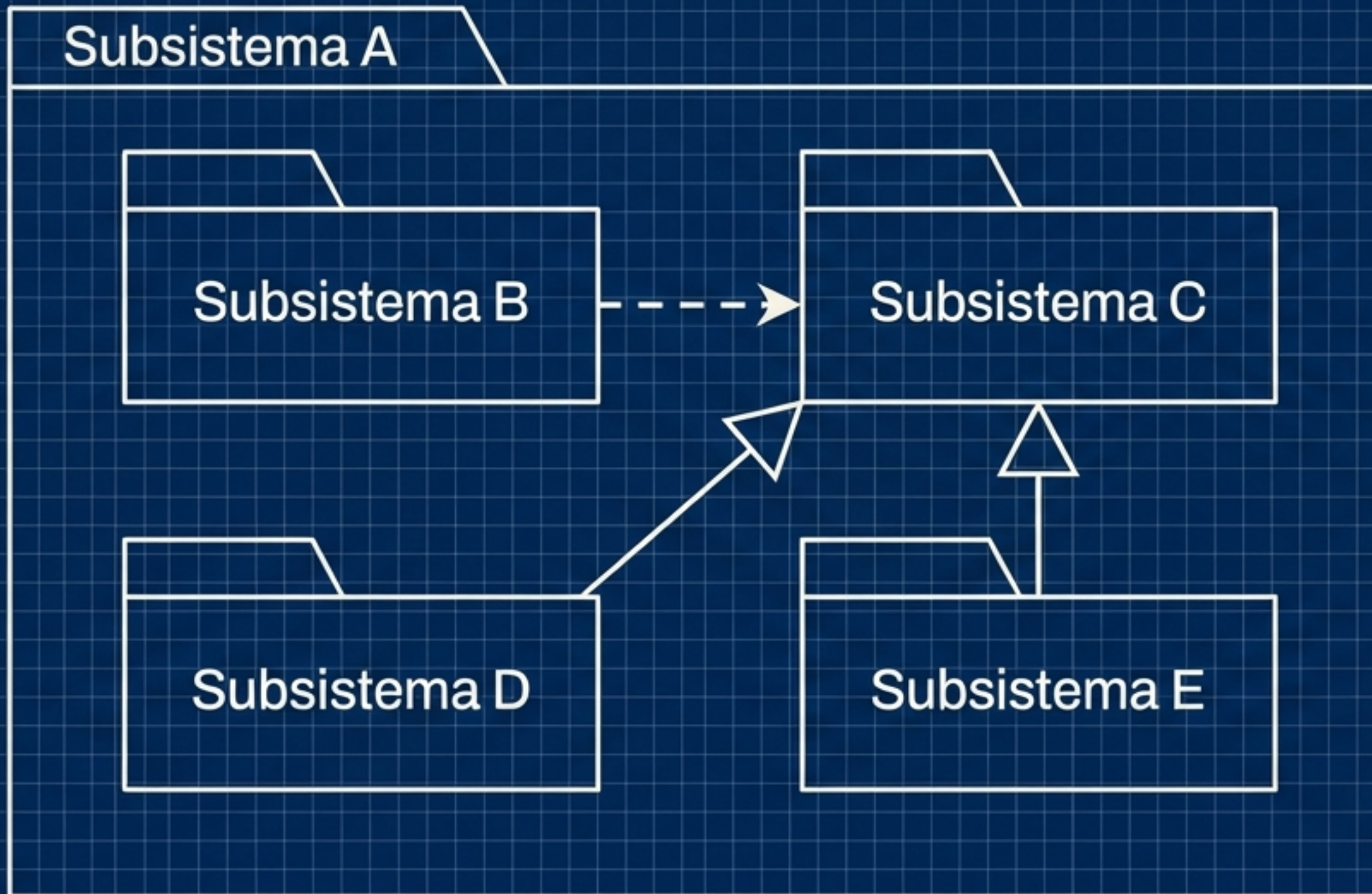


- **Restricciones { }:** Reglas lógicas.
- **Atributos Derivados (/):** Calculados a partir de otros datos.
- **Estereotipos << >>:** Extensión del vocabulario UML.

```
{edad = fecha-actual - fecha-de-cumpleaños}
```

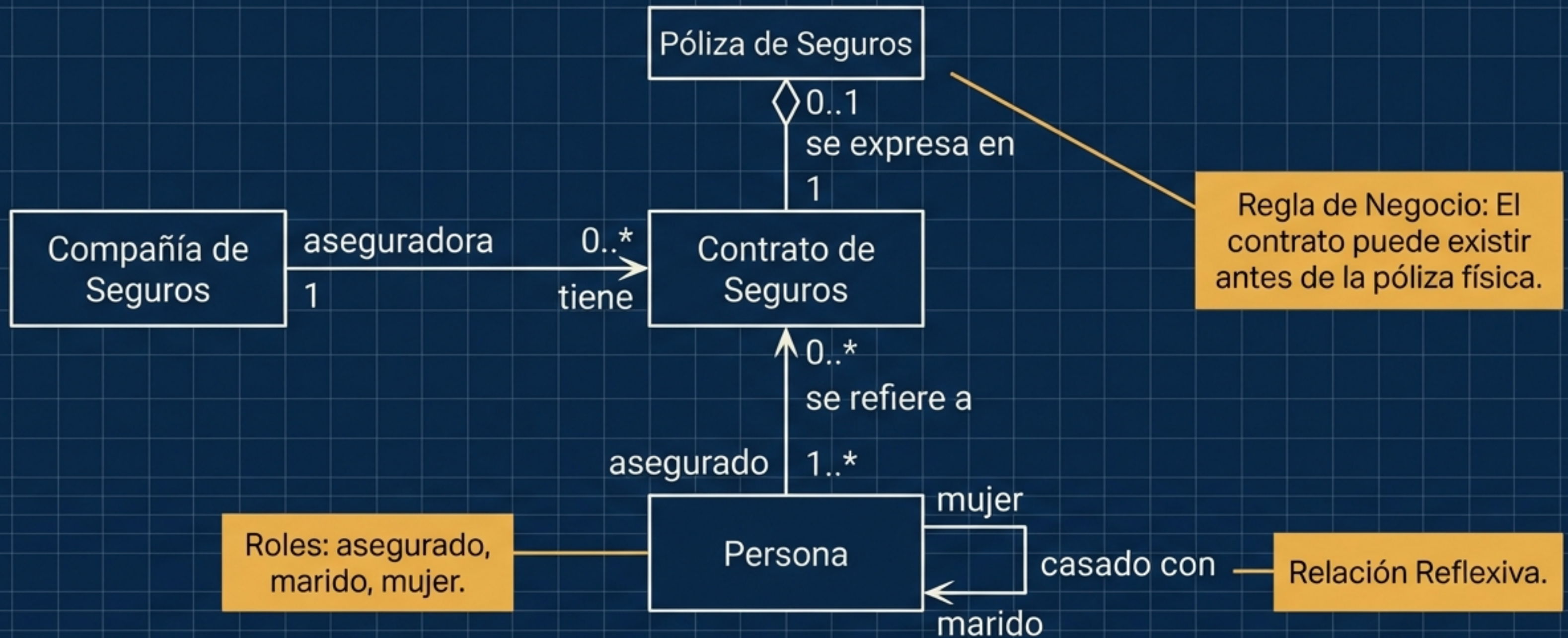
A note box connected to the 'edad' attribute of the 'Profesor' class. It contains the logical constraint: `{edad = fecha-actual - fecha-de-cumpleaños}`.

Organización a Gran Escala: Paquetes



Los paquetes agrupan elementos en subsistemas lógicos, permitiendo manejar la complejidad y definir espacios de nombres.

Integración: El Modelo de Dominio



Buenas Prácticas de Diseño

1

KISS (Keep It Simple): No modele todo. Céntrese en las áreas críticas. ✓



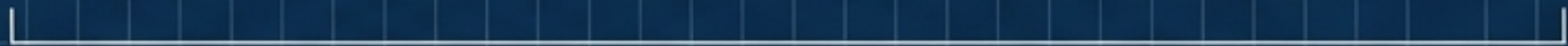
2

Claridad Visual: Minimice el cruce de líneas. Agrupe visualmente lo relacionado. ✓



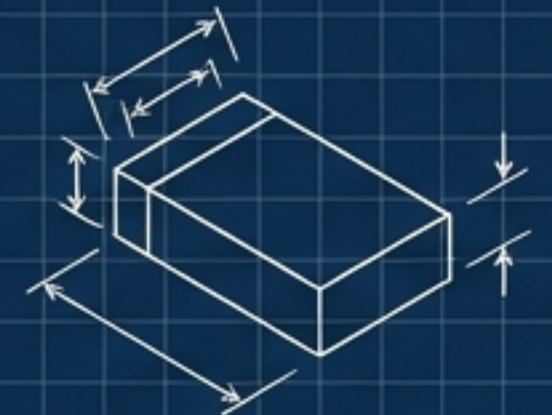
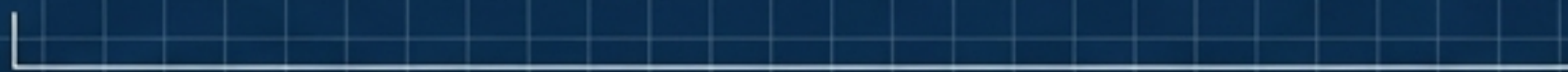
3

Abstracción Correcta: Elija la perspectiva adecuada (Conceptual vs. Implementación). ✓



4

Consistencia: Mantenga un estilo uniforme en nombres y visibilidad. ✓



Un buen diseño previene un sistema roto.
El diagrama es el mapa; el código es el territorio.

